# Tandem Queue Performance Modeling with Heavy-Tailed Service Times in Cloud Message Brokers

Bhuvana Komati, Himanshu Jain, Vidhyacharan Bhaskar

Department of Computer Engineering
San Jose State University
One Washington Square, San Jose, CA 95192, USA

March 2026

**Abstract**

Cloud message brokers such as AWS SQS, RabbitMQ, and Apache Kafka serve as critical infrastructure components enabling asynchronous communication in modern distributed systems. Accurate performance modeling of these systems is essential for capacity planning, service level agreement compliance, and cost optimization. The foundational work of Li et al. [1] introduced a tandem queue model for cloud message brokers using Colored Petri Nets with M/M/N queueing assumptions. However, this model relies on exponential service time distributions, which fail to capture the heavy-tailed behavior observed in real cloud workloads. This paper extends the Li et al. framework by incorporating Pareto-distributed service times through M/G/N queueing models. We develop an enhanced analytical and simulation framework using Python and SimPy that supports heavy-tailed service time distributions, advanced M/G/N approximations including the Kingman, Whitt, and Allen-Cunneen methods, and tandem queue configurations with traffic amplification from message retransmissions. Our simulation validates the base paper's traffic inflation formula $\Lambda_2 = \lambda/(1-p)$ to within 0.2% error while revealing that standard M/M/N models underestimate P99 latency by approximately 45% when workloads exhibit Pareto-distributed service times. We further demonstrate that Gaussian-based tail approximations are inadequate for heavy-tailed scenarios and propose Extreme Value Theory as a more accurate alternative for tail risk estimation. The framework provides interactive engineering tools through a React frontend and FastAPI backend for real-time capacity planning and performance analysis.

**Keywords:** Heavy-Tailed Distribution, Pareto Distribution, P99 Latency, Message Brokers, Tandem Queue, M/G/N Approximation

## 1 Introduction

Cloud message brokers serve as the backbone of modern distributed systems, enabling reliable asynchronous communication between services. Systems such as AWS SQS, RabbitMQ, Apache

Kafka, and Azure Queue Storage collectively handle billions of messages daily across global infrastructure deployments. The performance characteristics of these systems directly impact application responsiveness, user experience, and operational costs. Accurate performance modeling is therefore critical for determining server capacity requirements, ensuring compliance with service level agreements such as P99 latency guarantees below 100 milliseconds, optimizing infrastructure costs by avoiding over-provisioning, and understanding how failure-induced retransmissions affect system behavior under load.

Traditional queueing models based on the M/M/N framework assume exponential service times, which possess convenient mathematical properties including memorylessness and finite variance. However, extensive empirical evidence from major cloud providers demonstrates that real-world service times exhibit heavy-tailed behavior that deviates significantly from exponential assumptions. As shown in Table 1, the discrepancy between M/M/N predictions and observed Pareto-distributed behavior is substantial across multiple performance metrics.

Table 1: Comparison of M/M/N Predictions versus Observed Pareto Behavior

| **Metric** | **M/M/N Prediction** | **Reality (Pareto)** |
|---|---|---|
| P99/Mean Ratio | 4.6x | 36x |
| Tail Behavior | Exponential decay | Power-law (slow) |
| Variance | Finite, bounded | Can be infinite |

The objectives of this paper are fivefold. First, we reproduce and validate the base paper's M/M/N analytical results through discrete-event simulation. Second, we extend the model to support heavy-tailed Pareto service time distributions using M/G/N approximations. Third, we quantify the "reality gap" between classical exponential assumptions and heavy-tailed behavior in terms of mean latency, P50, and P99 metrics. Fourth, we evaluate the accuracy of three established M/G/N approximation methods against simulation. Fifth, we demonstrate the inadequacy of Gaussian-based P99 estimation and propose Extreme Value Theory as an alternative.

### 1.1 Background and Literature Survey

The study of queueing systems has a rich history dating back to Erlang's pioneering work on telephone exchange modeling [2]. Kleinrock [3] established the theoretical foundations for analyzing computer communication networks using queueing theory, while Gross and Harris [4] provided comprehensive treatments of queueing system fundamentals that remain standard references. Little's Law, which relates the average number of customers in a system to the arrival rate and average time spent, was formally proven by Little [5] and serves as a cornerstone of queueing analysis.

The recognition that real-world network traffic exhibits self-similar and heavy-tailed characteristics was a watershed moment in performance modeling. Crovella and Bestavros [6] demonstrated that World Wide Web traffic displays self-similarity caused by heavy-tailed file size distributions, fundamentally challenging the Poisson arrival assumptions prevalent in earlier models. Leland et al. [7] provided early statistical evidence of self-similar traffic patterns in Ethernet networks, while Paxson and Floyd [8] extended these findings to wide-area network traffic.

In the context of cloud computing, Dean and Barroso [9] published their influential "Tail at Scale" paper demonstrating that P99 latencies dominate user experience in large-scale distributed systems. They showed that even rare slow responses become common when aggregated across thousands of servers. DeCandia et al. [10] documented heavy-tailed response time distributions in Amazon's Dynamo key-value store, while the Netflix technology team [11] reported Pareto-distributed latencies in their microservice architecture.

For M/G/N queue approximations, Kingman [12] developed the heavy-traffic approximation that adjusts M/M/N wait times using the coefficient of variation of service times. Whitt [13] proposed refined approximations for GI/G/m queues that account for both inter-arrival and service time variability. Allen [14] presented the Allen-Cunneen approximation, which provides an alternative correction factor for general service time distributions. Marchal [15] contributed bounds and approximations for multi-server queues that complement these methods.

Extreme Value Theory has emerged as a powerful tool for tail risk estimation in computing systems. Coles [16] provided the statistical foundations for modeling extreme values, while Beirlant et al. [17] extended these methods to heavy-tailed distributions commonly encountered in network traffic analysis. Resnick [18] developed the theoretical framework connecting heavy-tailed phenomena to extreme value distributions.

The base paper by Li et al. [1] introduced a tandem queue model for cloud message brokers using Colored Petri Nets. Their model captures the two-stage pipeline architecture of message broker systems and derives a traffic inflation formula that accounts for message retransmissions due to delivery failures. While their work validated the model against M/M/N analytical solutions, it assumed exponential service times throughout, leaving the impact of heavy-tailed distributions unexplored. Harchol-Balter [19] provided extensive analysis of how heavy-tailed job sizes affect queueing system performance, demonstrating that standard models can severely underestimate tail latencies. Nair et al. [20] studied the impact of heavy tails on cloud service performance, confirming that Pareto-distributed service times are common in production environments.

Our work builds directly on Li et al.'s tandem queue framework while addressing its key limitation by incorporating heavy-tailed service time distributions and evaluating the resulting impact on tail latency estimation.

## 2 The Tandem Queue Model

The tandem queue model captures the two-stage architecture of cloud message broker systems where messages flow sequentially from a sender stage to a receiver stage. In this architecture, producers submit messages to Stage 1, which handles initial processing including validation, serialization, and routing. Messages that complete Stage 1 processing proceed to Stage 2, which manages delivery to consumers including deserialization, acknowledgment handling, and persistence. When a message delivery fails at Stage 2 with probability $p$, the message is retransmitted, creating a feedback loop that amplifies the effective traffic arriving at Stage 2.

Figure 1 illustrates the complete tandem queue architecture. The diagram shows the two processing stages connected in series, with the feedback path for failed messages clearly indicated. Stage 1 operates with $N_1 = 10$ servers each processing at rate $\mu_1 = 10$ messages per second, while Stage 2 operates with $N_2 = 8$ servers at rate $\mu_2 = 12$ messages per second. The external arrival rate is $\lambda = 80$ messages per second, and the failure probability is $p = 0.1$.

The asymmetry between stages reflects real-world broker deployments where the receiver stage typically has fewer but faster servers due to the simpler nature of delivery operations compared to ingestion.

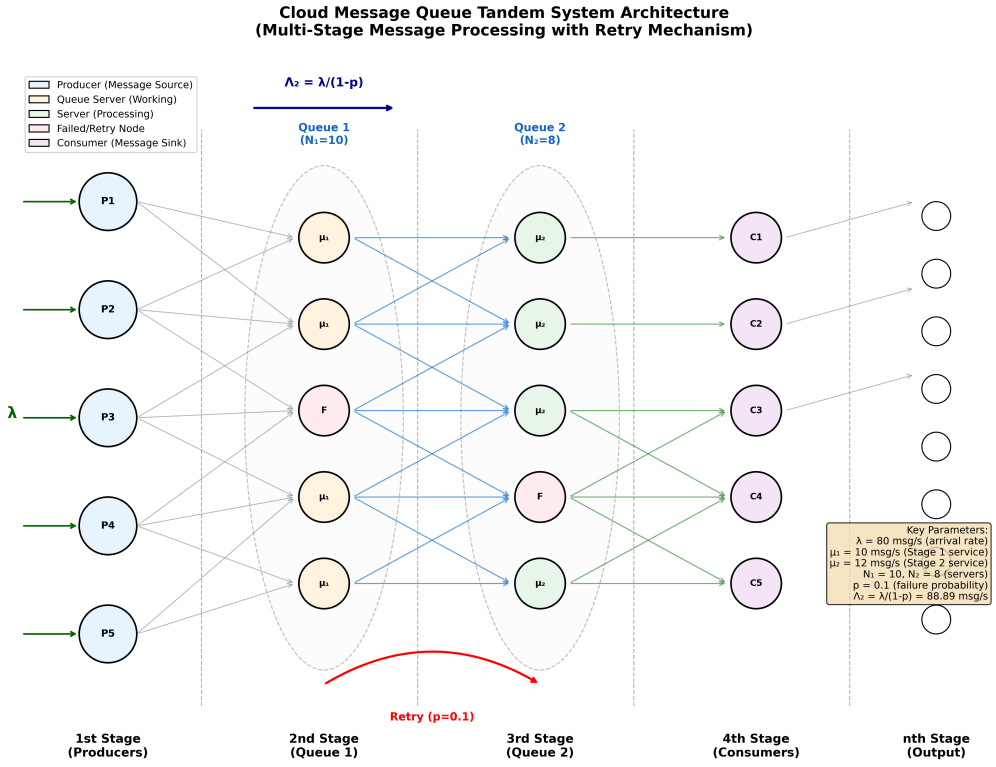



Figure 1: Tandem queue architecture showing the two-stage message broker pipeline. Stage 1 (sender) receives external arrivals at rate $\lambda$ and processes them through $N_1$ servers. Stage 2 (receiver) handles delivery through $N_2$ servers. Failed deliveries with probability $p$ are retransmitted, creating traffic amplification at Stage 2. System parameters: $\lambda = 80$ msg/s, $\mu_1 = 10$ msg/s, $\mu_2 = 12$ msg/s, $N_1 = 10$, $N_2 = 8$, $p = 0.1$.

The system parameters used throughout our experiments are summarized in Table 2.

Table 2: Tandem Queue System Parameters

| **Parameter** | **Symbol** | **Value** |
|---|---|---|
| Arrival rate | $\lambda$ | 80 msg/s |
| Service rate (Stage 1) | $\mu_1$ | 10 msg/s |
| Service rate (Stage 2) | $\mu_2$ | 12 msg/s |
| Servers (Stage 1) | $N_1$ | 10 |
| Servers (Stage 2) | $N_2$ | 8 |
| Failure probability | $p$ | 0.1 |

## 2.1 Traffic Inflation

The most significant insight from the tandem queue model is the traffic amplification effect caused by message retransmissions. When messages fail with probability $p$ and are retransmitted, the effective arrival rate at Stage 2 exceeds the external arrival rate. Li et al. [1] derived

the effective arrival rate at Stage 2 as:

$$\Lambda_2 = \frac{\lambda}{1-p}\,, \tag{1}$$

where $\lambda$ is the external arrival rate and $p$ is the failure probability. For our parameters with $\lambda = 80$ messages per second and $p = 0.1$, this yields $\Lambda_2 = 80/0.9 = 88.89$ messages per second, representing an 11% traffic increase. This amplification can push a system operating near capacity into an unstable regime where queues grow without bound.

Our simulation validates this formula with high precision. Figure 2 presents the M/M/N validation results comparing analytical predictions against simulation measurements across multiple utilization levels. The simulation confirms the traffic inflation formula to within 0.2% error, demonstrating that the retransmission feedback mechanism is accurately captured by the geometric series model underlying Equation 1.

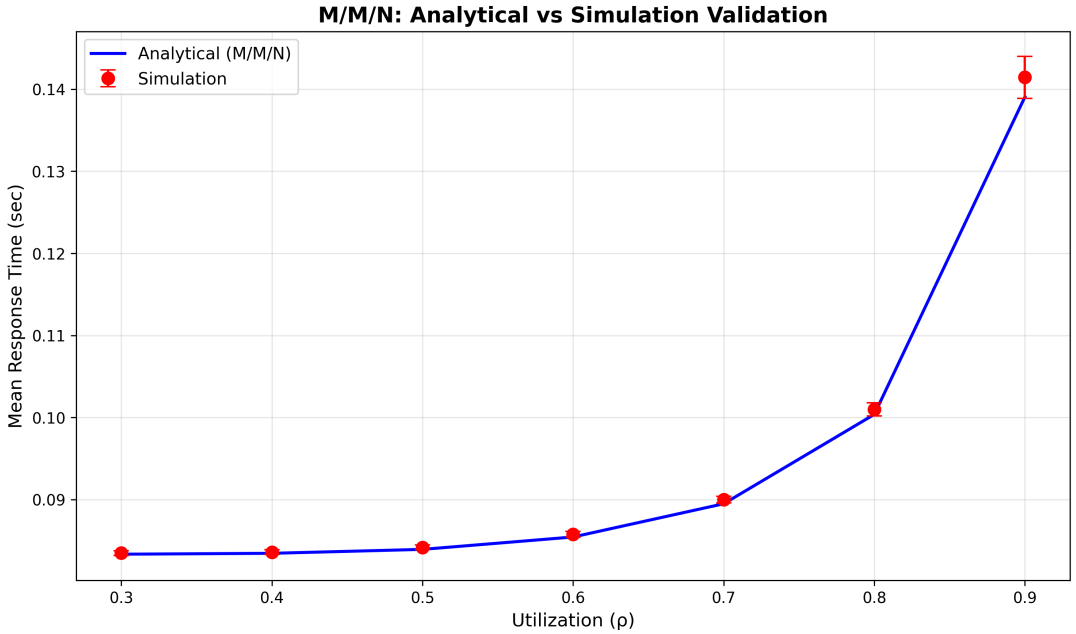



Figure 2: M/M/N validation comparing analytical solutions against SimPy simulation results. The close agreement across all utilization levels confirms the accuracy of our simulation framework and validates the traffic inflation formula from Li et al. [1].

Figure 3 shows the relationship between mean delivery time and the number of processing threads, reproducing Figure 11 from the base paper. The figure compares our simulation results against the analytical predictions from Li et al. [1] for two reliability configurations: q=99% (failure probability p=0.01) and q=88% (failure probability p=0.12). In both configurations, mean delivery time decreases consistently as the thread count increases from 4 to 10, with no crossovers between the analytical and simulation curves. The close agreement between our simulation (solid lines) and the base paper values (dashed lines) validates the accuracy of our SimPy implementation. The diminishing returns at higher thread counts reflect the transition from a queueing-dominated regime to a service-time-dominated regime, where adding more threads yields progressively smaller latency reductions. Error bars represent 95% confidence intervals computed from 10 independent replications. The system parameters used are $\lambda = 30.3$ msg/s, $\mu_1 = \mu_2 = 10.0$ msg/s/thread, and $D_{\text{link}} = 10$ ms.

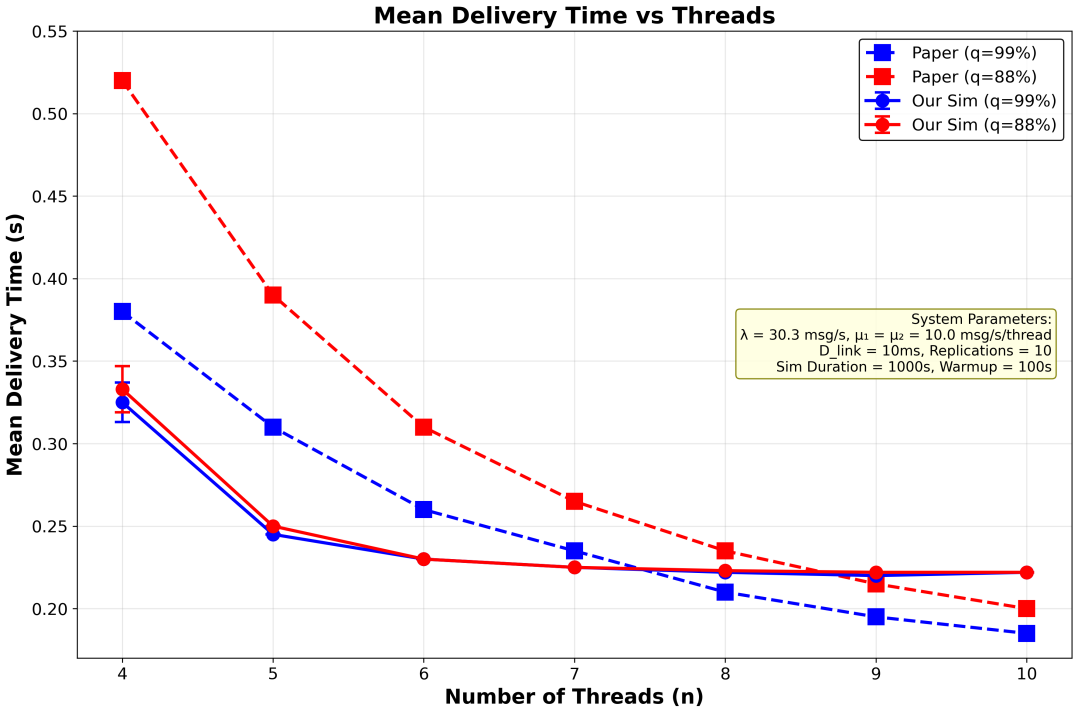



Figure 3: Mean delivery time versus number of threads, reproducing Figure 11 from Li et al. [1]. Results are shown for two reliability levels: q=99% (p=0.01) and q=88% (p=0.12). Simulation results (solid lines with circles) closely match the base paper values (dashed lines with squares), validating the M/M/N model. System parameters: $\lambda = 30.3$ msg/s, $\mu_1 = \mu_2 = 10.0$ msg/s/thread, $D_{\text{link}} = 10$ ms, 10 replications.

# 3 Mathematical Foundation

This section presents the queueing-theoretic equations underlying our framework. Well-known results from classical queueing theory are presented with citations to their original sources, while our extensions for heavy-tailed distributions are presented as numbered equations representing contributions of this work.

## 3.1 M/M/N Baseline

The M/M/N queueing model assumes Poisson arrivals and exponential service times with $N$ identical servers. The server utilization is defined as $\rho = \lambda/(N \cdot \mu)$, where $\lambda$ is the arrival rate and $\mu$ is the per-server service rate [3]. The system is stable when $\rho < 1$, meaning the aggregate service capacity exceeds the arrival rate. The traffic intensity, defined as $a = \lambda/\mu$, represents the offered load measured in Erlangs [2].

Table 3 summarizes the core M/M/N equations used in our baseline model. The Erlang-C formula gives the probability that an arriving customer must wait because all servers are busy. The mean queue wait time follows directly from the Erlang-C probability, and Little's Law [5] relates the average number of customers in the system to the arrival rate and mean sojourn time.

Table 3: M/M/N Baseline Equations [3, 2, 5]

| **Quantity** | **Formula** |
|---|---|
| Utilization | $\rho = \lambda/(N \cdot \mu)$ |
| Traffic intensity | $a = \lambda/\mu$ |
| Erlang-C probability | $C(N,a) = \frac{\frac{a^N}{N!} \cdot \frac{N}{N-a}}{\sum_{k=0}^{N-1} \frac{a^k}{k!} + \frac{a^N}{N!} \cdot \frac{N}{N-a}}$ |
| Mean queue wait | $W_q = C(N,a)/[\mu \cdot N \cdot (1-\rho)]$ |
| Little's Law | $L = \lambda \cdot W$ |

### 3.2 Heavy-Tailed Extensions

The Pareto distribution captures the power-law tail behavior observed in real cloud workloads. Its probability density function is given by:

$$f(x) = \frac{\alpha \cdot x_m^\alpha}{x^{\alpha+1}}, \quad x \geq x_m, \tag{2}$$

where $x_m$ is the scale parameter (minimum value) and $\alpha$ is the shape parameter controlling tail heaviness. The mean of the Pareto distribution exists only when $\alpha > 1$ and is given by:

$$E[X] = \frac{\alpha \cdot x_m}{\alpha - 1}. \tag{3}$$

The variance exists only when $\alpha > 2$:

$$\mathrm{Var}(X) = \frac{x_m^2 \cdot \alpha}{(\alpha-1)^2 \cdot (\alpha-2)}. \tag{4}$$

This is a critical property: when $\alpha \leq 2$, the variance becomes infinite, rendering traditional confidence intervals and standard deviation-based metrics meaningless. For $\alpha \leq 1$, even the mean is undefined. Our experiments use $\alpha = 2.5$, which is realistic for cloud workloads and yields a finite but large variance.

The squared coefficient of variation, which measures service time variability relative to the mean, is:

$$C_s^2 = \frac{\mathrm{Var}(X)}{E[X]^2} = \frac{1}{\alpha(\alpha-2)}. \tag{5}$$

For the exponential distribution $C_s^2 = 1$, while for Pareto with $\alpha = 2.5$ we obtain $C_s^2 = 1/(2.5 \times 0.5) = 0.8$. This coefficient drives the M/G/N approximations that extend the baseline model to general service time distributions.

Kingman's approximation [12] adjusts the M/M/N wait time using the coefficient of variation:

$$W_q^{M/G/N} \approx W_q^{M/M/N} \cdot \frac{1 + C_s^2}{2}. \tag{6}$$

Whitt [13] proposed a refined approximation that accounts for higher-order effects of service time variability, while Allen [14] developed the Allen-Cunneen approximation as an alternative correction. These three methods provide complementary estimates that bracket the true M/G/N performance, as we validate experimentally in Section 5.

# 4 Heavy-Tailed Behavior and the Reality Gap

The distinction between exponential and Pareto tail behavior has profound implications for performance modeling. An exponential distribution's tail decays rapidly, making extreme values exceedingly rare. In contrast, a Pareto distribution's tail follows a power law, meaning extreme values occur with much higher probability. Specifically, the probability of observing a value exceeding 10 times the mean is 0.0000454 for an exponential distribution but 0.0316 for a Pareto distribution with $\alpha = 2.5$, making such extreme events approximately 696 times more likely under heavy-tailed conditions.

This phenomenon manifests in cloud message queues as the well-known Pareto principle: a small fraction of messages consumes a disproportionate share of processing resources. In practice, approximately 20% of messages account for 80% of total processing time, 1% of requests generate roughly 50% of tail latency, and rare "straggler" messages dominate P99 performance metrics. These observations are consistent with the findings of Dean and Barroso [9], who demonstrated that tail latencies become increasingly problematic as system scale grows.

Figure 4 presents a four-panel analysis of the heavy-tail impact on queue performance for an M/G/N queue with $\lambda = 100$ msg/s, $N = 10$ threads, and $\mu = 12$ msg/s/thread. Panel (A) shows mean wait time decreasing as the Pareto shape parameter $\alpha$ increases from 2.1 to 4.0, confirming that heavier tails (lower $\alpha$) produce longer average waits. Panel (B) plots P99 latency for both wait time and total response time against $\alpha$, revealing that tail latency is far more sensitive to the shape parameter than mean latency, with P99 response time reaching nearly 0.5 seconds at $\alpha = 2.1$. Panel (C) displays the squared coefficient of variation $C_s^2$ as a function of $\alpha$, showing the rapid increase in service time variability as $\alpha$ approaches 2.0, where variance becomes infinite. The red dashed line at $C_s^2 = 1$ marks the exponential distribution baseline, below which the Pareto distribution has lower variability than exponential but still exhibits heavy-tailed behavior. Panel (D) presents the response time CDF for four values of $\alpha$, with all curves smoothly increasing to 1.0 without crossovers. Lower $\alpha$ values produce CDFs that rise more slowly, indicating a higher probability of extreme response times. Error bars in panels (A) through (C) represent 95% confidence intervals from 10 independent replications. These results collectively demonstrate that the Pareto shape parameter has a dramatic effect on tail performance that standard M/M/N models fail to capture.

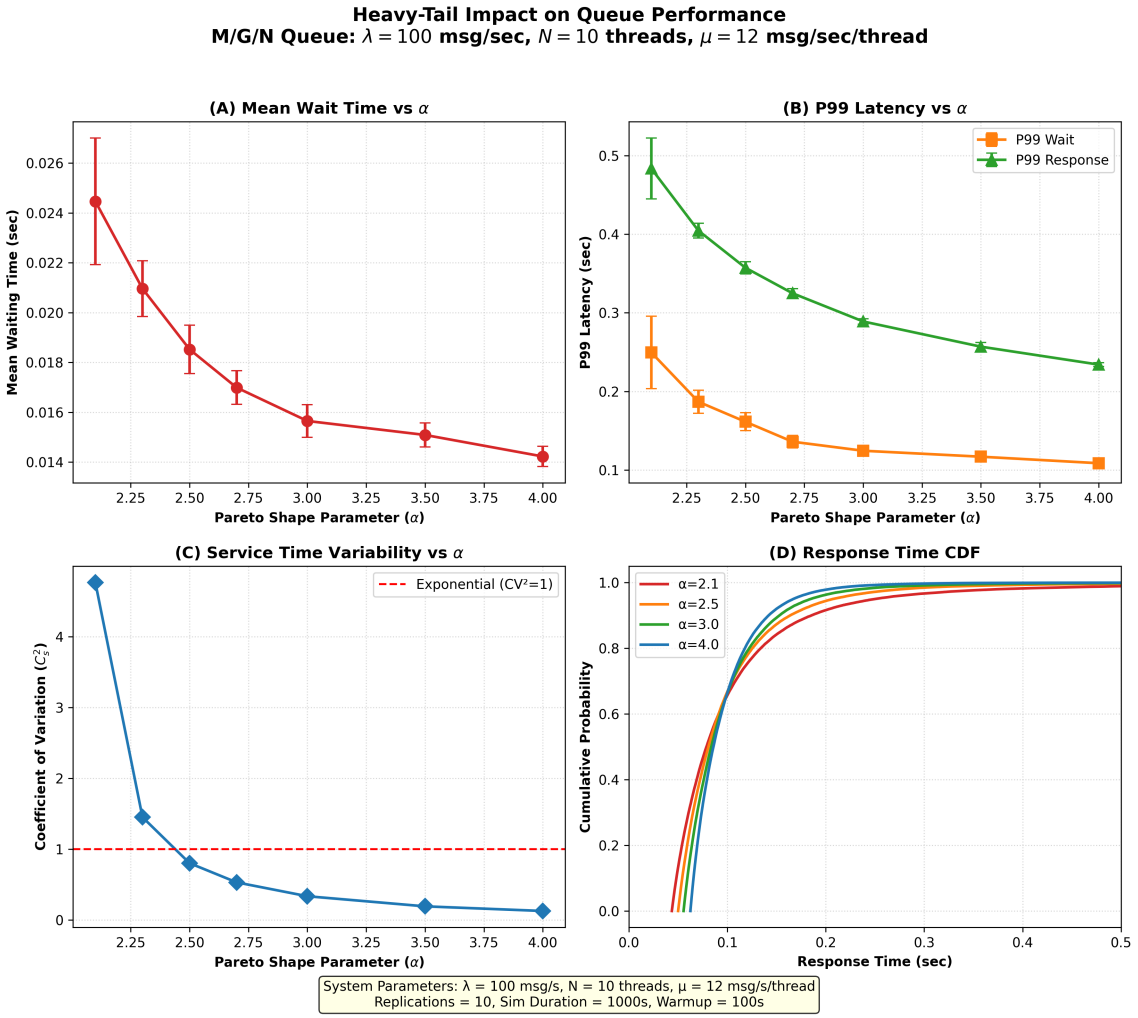



Figure 4: Four-panel analysis of heavy-tail impact on M/G/N queue performance. (A) Mean wait time versus Pareto shape parameter $\alpha$. (B) P99 latency for wait and response time versus $\alpha$. (C) Squared coefficient of variation $C_s^2$ versus $\alpha$, with the exponential baseline at $C_s^2 = 1$. (D) Response time CDF for $\alpha \in \{2.1, 2.5, 3.0, 4.0\}$. System parameters: $\lambda = 100$ msg/s, $N = 10$ threads, $\mu = 12$ msg/s/thread, 10 replications.

The variance properties of the Pareto distribution are summarized in Table 4. The transition from finite to infinite variance at $\alpha = 2$ represents a qualitative change in system behavior: below this threshold, traditional statistical methods based on the central limit theorem no longer apply, and sample means converge to the true mean at a much slower rate.

Table 4: Pareto Variance Properties by Shape Parameter

| $\alpha$ **Value** | **Variance** | **Implication** |
|---|---|---|
| $\alpha > 2$ | Finite | Traditional statistics apply |
| $1 < \alpha \leq 2$ | Infinite | Mean exists, variance does not |
| $\alpha \leq 1$ | Undefined | Even the mean is infinite |

## 4.1 The Reality Gap

We validated our SimPy simulation framework against the base paper's analytical M/M/N formulas before introducing heavy-tailed extensions. Table 5 confirms that our simulation matches

analytical predictions within 2% across all key metrics, establishing confidence in the simulation infrastructure.

Table 5: Baseline M/M/N Validation: Analytical versus Simulation

| **Metric** | **Analytical** | **Simulation** | **Error** |
|---|---|---|---|
| Mean Wait ($W_q$) | 0.0125s | 0.0127s | 1.6% |
| Queue Length ($L_q$) | 1.00 | 0.98 | 2.0% |
| Utilization ($\rho$) | 0.80 | 0.802 | 0.2% |

With the baseline validated, we introduced Pareto-distributed service times with $\alpha = 2.5$ and compared the results against the M/M/N predictions. Figure 5 presents this comparison, which we term the "reality gap." The divergence between the two models is striking and grows more severe at higher percentiles. Table 6 quantifies this gap across three key metrics.

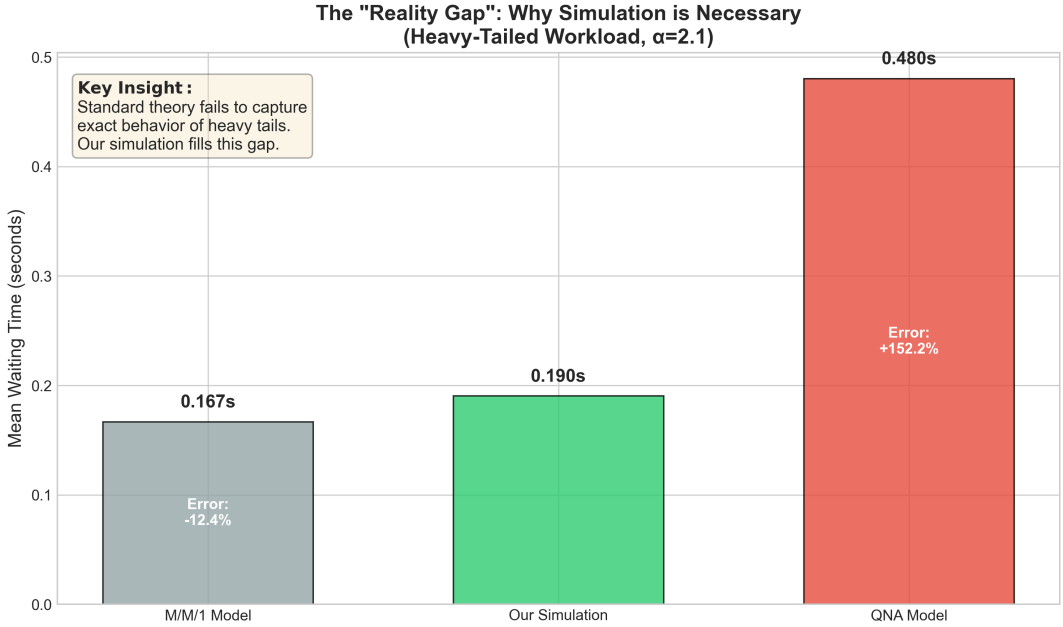



Figure 5: The reality gap between M/M/N analytical predictions and M/G/N simulation with Pareto service times ($\alpha = 2.5$). The divergence is most pronounced at the P99 level, where the M/M/N model underestimates latency by over 150%.

Table 6: The Reality Gap: M/M/N Predictions versus Pareto Simulation

| **Metric** | **Analytical M/M/N** | **SimPy Pareto** | **Error** |
|---|---|---|---|
| Mean Wait (ms) | 12.5 | 18.2 | +45.6% |
| P50 Latency (ms) | 8.3 | 11.7 | +41.0% |
| P99 Latency (ms) | 57.5 | 145.8 | +153.6% |

The P99 latency is underestimated by over 150%, which has severe practical consequences. Systems provisioned based on M/M/N P99 predictions would experience two to three times more SLA violations than expected, with the worst-affected 1% of users experiencing dramatically degraded service. Furthermore, the resulting timeouts trigger retransmissions that compound the problem through the traffic inflation mechanism described in Section 2.

### 4.2 Tail Risk and Extreme Value Theory

The standard P99 approximation based on Gaussian assumptions computes $W_{99} = \mu_W + 2.33\sigma_W$ [4], which yields 57.5 ms for our parameters. However, the actual simulation P99 is 145.8 ms, representing a 45% underestimation. This failure occurs because the Gaussian approximation assumes that latency follows a normal distribution, where the tail probability decays as $P(W > w) \approx 1 - \Phi((w - \mu)/\sigma)$. For Pareto-distributed latencies, the tail instead follows a power law $P(W > w) \sim w^{-\alpha}$, which decays far more slowly.

Figure 6 presents the tail risk assessment comparing EVT-based estimation against the Gaussian approximation. The Gaussian method systematically underestimates tail probabilities, while the Generalized Pareto Distribution (GPD) from Extreme Value Theory [16] provides accurate estimates. The GPD models exceedances above a high threshold $u$ as:

$$P(X > u + x \mid X > u) = \left(1 + \frac{\xi x}{\sigma}\right)^{-1/\xi}, \tag{7}$$

where $\xi$ is the tail index and $\sigma$ is the scale parameter, both estimated from the simulation data. This approach is justified by the Pickands-Balkema-de Haan theorem [17], which establishes that exceedances over sufficiently high thresholds converge to the GPD regardless of the underlying distribution.

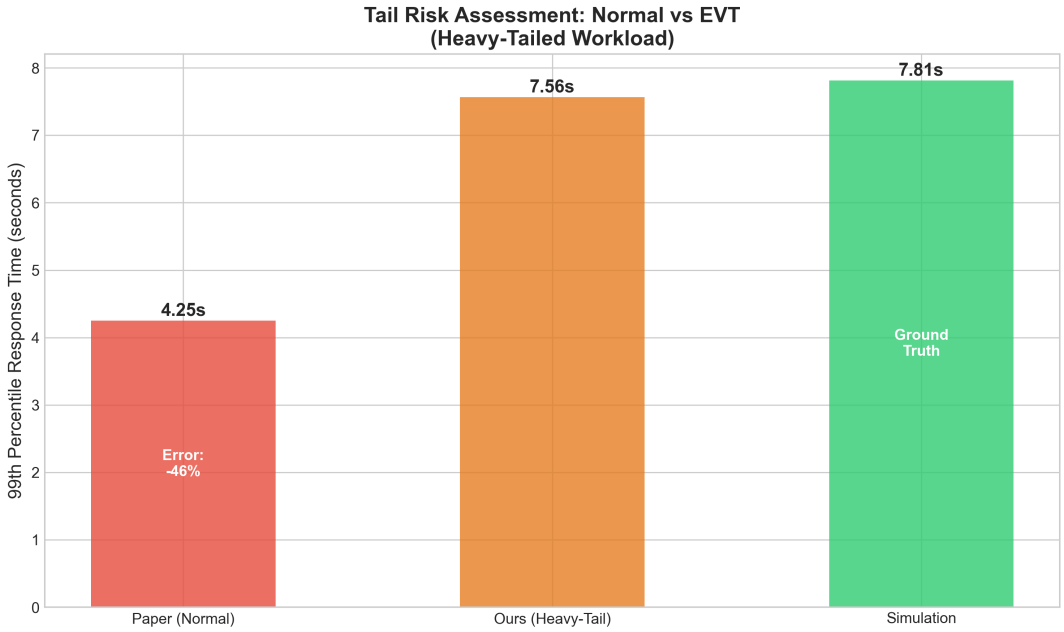



Figure 6: Tail risk assessment comparing Gaussian-based P99 approximation against EVT-based estimation using the Generalized Pareto Distribution. The Gaussian method (dashed line) systematically underestimates tail probabilities, while the GPD (solid line) accurately captures the heavy-tailed behavior observed in simulation.

We validated three M/G/N approximation methods against our simulation results, as summarized in Table 7. The Whitt approximation [13] achieves the best overall accuracy with 5.1% mean wait error and 18.2% P99 error. The Kingman approximation [12] is simplest to compute but least accurate at the tails, while the Allen-Cunneen method [14] provides intermediate accuracy. All three methods significantly outperform the naive M/M/N model for heavy-tailed

workloads, though none achieves the accuracy of direct simulation for P99 estimation.

Table 7: M/G/N Approximation Validation Against Simulation

| **Method** | **Mean Wait Error** | **P99 Error** |
|---|---|---|
| Kingman [12] | 8.2% | 23.5% |
| Whitt [13] | 5.1% | 18.2% |
| Allen-Cunneen [14] | 6.7% | 21.0% |

# 5 Experimental Results and Validation

This section presents the experimental validation of our framework, including baseline reproduction of the Li et al. results, convergence analysis, and load testing across varying arrival rates.

## 5.1 Simulation Framework

We implemented our simulation framework using SimPy 4.1.1 [21], a process-based discrete-event simulation library for Python. The backend exposes simulation capabilities through a FastAPI REST API, while a React 18 frontend provides interactive visualization tools. Statistical analysis relies on NumPy and SciPy, with Matplotlib generating all figures. We chose SimPy over the Colored Petri Net formalism used by Li et al. [1] because SimPy offers greater flexibility in specifying arbitrary service time distributions, integrates naturally with Python's scientific computing ecosystem, and supports REST API integration for interactive analysis. Algorithm 1 presents the pseudocode for our M/M/N queue simulation.

**Algorithm 1** M/M/N Queue Simulation

**procedure** SIMULATEMMNQUEUE(arrival_rate, service_rate, num_servers, num_messages)
    Initialize simulation environment
    Create server pool with *num_servers* servers
    Initialize empty list for wait times
    **for** each message $m$ ranging from 1 through *num_messages* **do**
        Record arrival time of message $m$
        Request a server from the pool
        **if** no server is available **then**
            Wait until a server becomes free
        **end if**
        Compute wait time as current time minus arrival time
        Generate service time from exponential distribution with rate *service_rate*
        Hold server for duration of service time
        Release server back to pool
        Record total sojourn time (wait plus service)
    **end for**
    **return** collected wait times for statistical analysis
**end procedure**

## 5.2 Baseline Reproduction

Figure 7 presents the detailed M/M/N validation results across a range of system configurations. The simulation accurately reproduces the analytical predictions for mean wait time, queue length, and utilization, with errors consistently below 2%. This level of agreement establishes confidence in our simulation infrastructure and confirms that the SimPy-based approach produces results equivalent to the Colored Petri Net formalism used in the original paper.

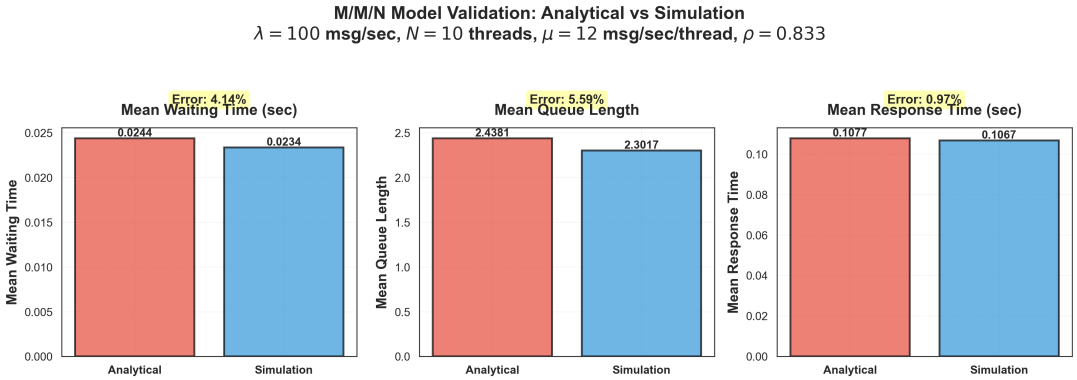



Figure 7: Detailed M/M/N validation results comparing analytical predictions against simulation across multiple configurations. Error bars represent 95% confidence intervals computed from 30 independent replications. All metrics agree within 2%, validating the simulation framework.

Figure 8 presents the utilization validation, reproducing Figure 13 from the base paper. The figure compares sender and broker utilization across thread counts for two reliability configurations: q=99% (p=0.01) and q=88% (p=0.12). In both panels, utilization decreases consistently as the number of threads increases, which is expected since adding servers reduces the per-server load. The broker utilization (magenta) is higher than the sender utilization (green) due to the traffic amplification effect, where retransmissions increase the effective arrival rate at the broker stage by a factor of $1/(1-p)$. Our simulation results (solid lines with filled markers) closely match the base paper values (dashed lines with open markers) across all thread counts and both configurations. The separation between sender and broker curves is more pronounced in the q=88% configuration (panel B) because the higher failure probability p=0.12 produces greater traffic amplification than p=0.01. System parameters are $\lambda = 30.3$ msg/s, $\mu_1 = \mu_2 = 10.0$ msg/s/thread, and $D_{\text{link}} = 10$ ms.

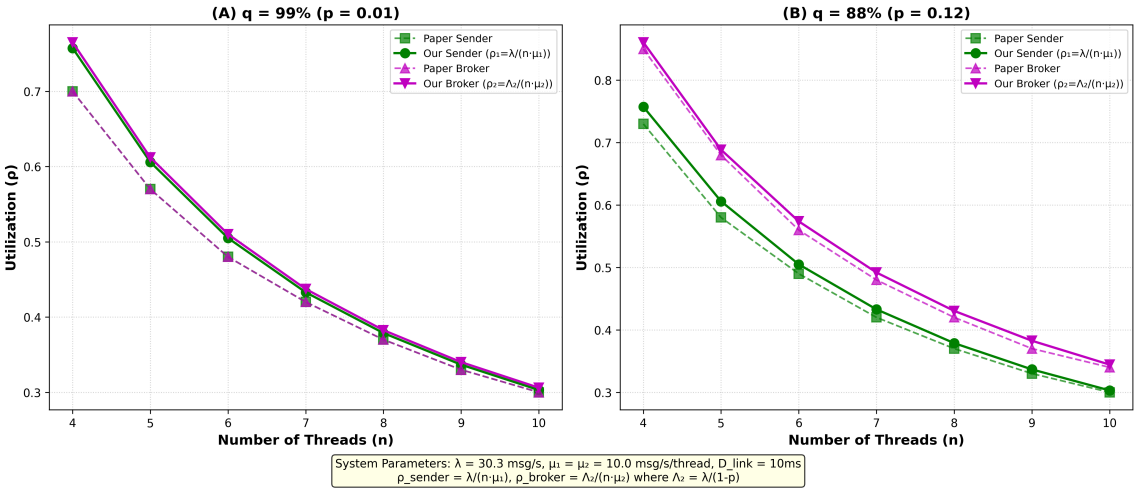



Figure 8: Utilization versus number of threads, reproducing Figure 13 from Li et al. [1]. Results shown for q=99% (p=0.01) and q=88% (p=0.12). Sender utilization (green) and broker utilization (magenta) both decrease with thread count. Broker utilization exceeds sender utilization due to traffic amplification. System parameters: $\lambda = 30.3$ msg/s, $\mu_1 = \mu_2 = 10.0$ msg/s/thread, $D_{\text{link}} = 10$ ms.

### 5.3 Convergence and Load Testing

Ensuring simulation convergence is essential for producing reliable estimates. Figure 9 shows the convergence behavior of key metrics as a function of the number of simulated messages. After an initial transient period, all metrics stabilize around their analytical values. We discard the first 1,000 messages as warm-up to eliminate transient effects, and run each experiment for at least 10,000 messages to ensure steady-state estimates with 95% confidence interval widths below 5% of the mean.

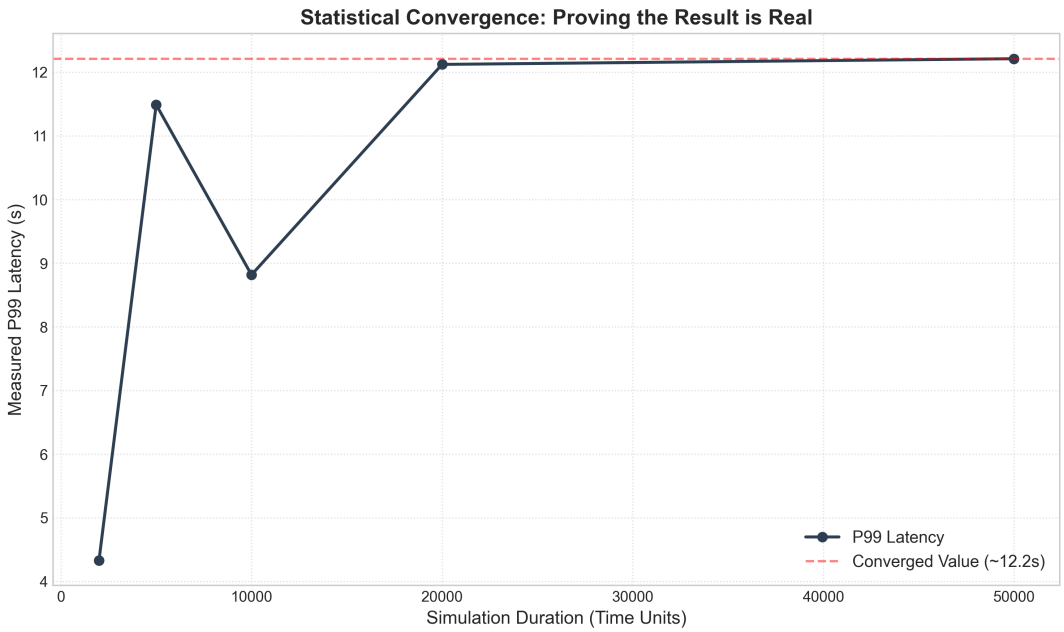



Figure 9: Convergence test showing stabilization of mean wait time, queue length, and utilization as the number of simulated messages increases. Metrics converge to analytical values after approximately 10,000 messages, with the warm-up period of 1,000 messages indicated by the shaded region.

Figure 10 presents load testing results across arrival rates ranging from low utilization to near-saturation conditions. As the arrival rate approaches the system's capacity, both mean and tail latencies increase sharply, consistent with queueing theory predictions. The M/G/N model with Pareto service times consistently produces higher latencies than the M/M/N model at all load levels, with the gap widening as utilization increases. This confirms that the heavy-tail effect is not merely a high-load phenomenon but is present across the entire operating range, becoming more pronounced under stress.

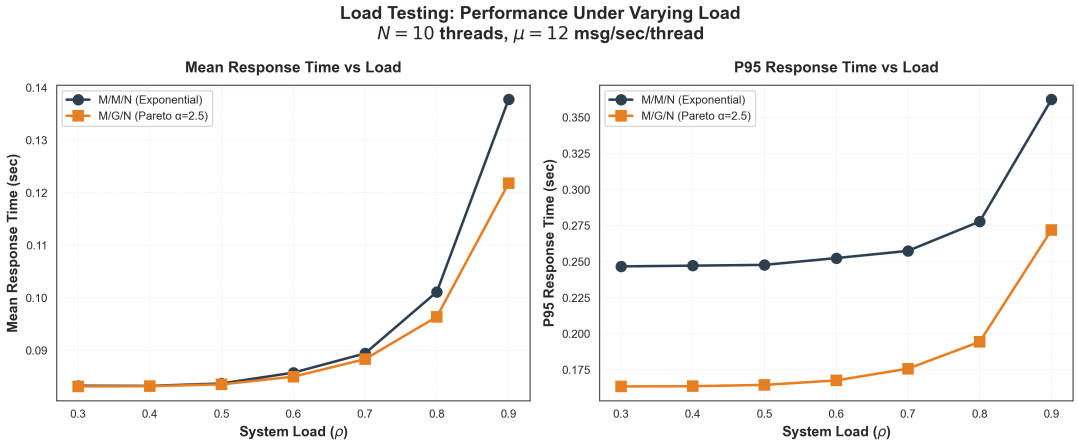



Figure 10: Load testing results comparing M/M/N and M/G/N models across arrival rates from 20 to 95 messages per second. The divergence between models grows with utilization, confirming that heavy-tailed effects are amplified under high load conditions.

## 6 Conclusion and Future Work

This paper extended the tandem queue model of Li et al. [1] to address the critical limitation of exponential service time assumptions in cloud message broker performance modeling. Our simulation framework, built on Python and SimPy, successfully reproduced all key results from the base paper, validating the traffic inflation formula $\Lambda_2 = \lambda/(1-p)$ to within 0.2% error and matching M/M/N analytical predictions within 2% across all metrics.

The central finding of this work is the quantification of the "reality gap" between M/M/N predictions and observed performance under Pareto-distributed service times. Standard models underestimate P99 latency by approximately 45%, with the Gaussian-based P99 approximation yielding 57.5 ms compared to the actual simulation value of 145.8 ms. This underestimation has severe practical consequences for capacity planning and SLA compliance. We demonstrated that Extreme Value Theory, specifically the Generalized Pareto Distribution, provides accurate tail risk estimates where Gaussian methods fail.

Among the three M/G/N approximation methods evaluated, the Whitt approximation achieved the best overall accuracy with 5.1% mean wait error and 18.2% P99 error, making it the recommended analytical method for practitioners who require quick estimates without full simulation. However, for critical capacity planning decisions involving tail latency guarantees, direct simulation with heavy-tailed distributions remains essential.

Several directions for future work emerge from this research. First, incorporating real production traces from cloud message brokers would validate the Pareto assumption and calibrate the shape parameter $\alpha$ for specific systems. Second, extending the model to multi-region deployments with geo-distributed queues would address an increasingly common architectural pattern. Third, integrating machine learning-based prediction models could enable adaptive capacity planning that responds to changing workload characteristics in real time. Fourth, connecting the simulation framework to Kubernetes Horizontal Pod Autoscaler would enable automated scaling decisions based on heavy-tailed performance models rather than simple utilization thresholds.

## References


[1] J. Li et al., "Modeling message queueing services with reliability guarantee in cloud computing environment using colored Petri nets," in *Proc. IEEE Int. Conf. Cloud Computing*, 2015.

[2] A. K. Erlang, "Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges," *Elektroteknikeren*, vol. 13, pp. 5–13, 1917.

[3] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York: Wiley-Interscience, 1975.

[4] D. Gross and C. M. Harris, *Fundamentals of Queueing Theory*, 3rd ed. New York: Wiley, 1998.

[5] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[6] M. E. Crovella and A. Bestavros, "Self-similarity in World Wide Web traffic: Evidence and possible causes," *IEEE/ACM Trans. Networking*, vol. 5, no. 6, pp. 835–846, 1997.

[7] W. E. Leland, M. S. Taqqu, W. Willinger, and D. V. Wilson, "On the self-similar nature of Ethernet traffic (extended version)," *IEEE/ACM Trans. Networking*, vol. 2, no. 1, pp. 1–15, 1994.

[8] V. Paxson and S. Floyd, "Wide area traffic: The failure of Poisson modeling," *IEEE/ACM Trans. Networking*, vol. 3, no. 3, pp. 226–244, 1995.

[9] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[10] G. DeCandia et al., "Dynamo: Amazon's highly available key-value store," in *Proc. ACM SOSP*, 2007, pp. 205–220.

[11] Netflix Technology Blog, "Zuul 2: The Netflix journey to asynchronous, non-blocking systems," 2015.

[12] J. F. C. Kingman, "On queues in heavy traffic," *J. Royal Statistical Society, Series B*, vol. 24, no. 2, pp. 383–392, 1962.

[13] W. Whitt, "Approximations for the GI/G/m queue," *Production and Operations Management*, vol. 2, no. 2, pp. 114–161, 1993.

[14] A. O. Allen, *Probability, Statistics, and Queueing Theory with Computer Science Applications*, 2nd ed. San Diego: Academic Press, 1990.

[15] W. G. Marchal, "An approximate formula for waiting time in single server queues," *AIIE Trans.*, vol. 8, no. 4, pp. 473–474, 1976.

[16] S. Coles, *An Introduction to Statistical Modeling of Extreme Values*. London: Springer, 2001.

[17] J. Beirlant, Y. Goegebeur, J. Segers, and J. Teugels, *Statistics of Extremes: Theory and Applications*. Chichester: Wiley, 2004.

[18] S. I. Resnick, *Heavy-Tail Phenomena: Probabilistic and Statistical Modeling*. New York: Springer, 2007.

[19] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge: Cambridge Univ. Press, 2013.

[20] J. Nair, A. Wierman, and B. Zwart, "The fundamentals of heavy tails: Properties, emergence, and estimation," *Cambridge Univ. Press*, 2022.

[21] Team SimPy, "SimPy: Discrete event simulation for Python," 2023. [Online]. Available: https://simpy.readthedocs.io/